# Lecture 01 Slide-by-Slide Bloom Preview

Introduction to Software Engineering and Object-Oriented Fundamentals

| | |
|---|---|
| **Bloom score** | 82 |
| **Dominant level** | apply |
| **Slides** | 18 |

## Bloom distribution

| | |
|---|---|
| remember | 8% |
| understand | 22% |
| apply | 38% |
| analyze | 26% |
| evaluate | 4% |
| create | 2% |

## Slide 01. Software Construction, Architecture and Engineering

Bloom level: analyze | Layout: title

Evidence: Activity requires analyzing past software challenges and identifying common patterns, moving beyond simple recall of experiences.

heading: Lecture 1: Introduction to Software Engineering and Object-Oriented Fundamentals

text: MSc Computer Science and Software Engineering - 5 ECTS

Activity: Pre-Lecture Reflection: Your Software Engineering Journey | Write down (1 min): What is the largest software system you've worked on? What made it challenging? Pair (2 min): Share your experience and identify one common challenge. This course will equip you to systematically address these challenges at industrial scale. | Activate prior knowledge and establish relevance by connecting personal experiences to course goals, moving from passive reception to active engagement and analysis of past challenges.

Speaker notes: Welcome students warmly. Launch the reflection activity immediately to activate prior knowledge and establish relevance. Listen during pair discussions for common themes like 'code became messy,' 'hard to add features,' or 'team coordination issues.' Briefly acknowledge these challenges and preview that this course provides systematic solutions through three pillars: Construction (OO fundamentals), Architecture (design patterns), and Engineering (processes). This activity transforms a passive title slide into an engaging entry point that connects student experiences to course objectives.

## Slide 02. Course Structure: Three Pillars

Bloom level: analyze | Layout: content

Evidence: Think-pair-share requires analyzing dependencies between pillars and articulating how mastering one enables work in another.

diagram: Three interconnected pillars labeled 'Construction', 'Architecture', and 'Engineering' supporting a platform labeled 'High-Quality Software Systems'. Use professional colors and show bidirectional arrows between pillars indicating their relationships.

bullets: Construction: OO fundamentals, inheritance, genericity; Architecture: Design patterns, large-scale structuring; Engineering: Agile methods, requirements, processes; Assessment: 50% portfolio + 50% written exam (min 45% each)

Activity: Think-Pair-Share: Pillar Dependencies | Think (1 min): Which pillar must come first and why? Pair (2 min): Discuss with a neighbor how the three pillars depend on each other. Share: Explain to the class how mastering one pillar enables work in another. | Move from remembering course structure to analyzing relationships between topics and articulating how they build on each other.

Speaker notes: After presenting the structure, facilitate the think-pair-share activity. Listen for insights like 'Construction provides building blocks for Architecture' or 'Engineering practices guide when to apply Construction techniques.' Push students to articulate dependencies rather than just list topics. This addresses the objective to 'articulate course structure' at understand/analyze level. Conclude by connecting their insights to the 35-35-55 hour distribution and how this prepares them for Advanced Software Engineering.

## Slide 03. What You'll Learn: Course Objectives

Bloom level: analyze | Layout: content

Evidence: Activity requires identifying connections between objectives and determining which pillar each belongs to, analyzing relationships rather than listing.

bullets: Apply OO techniques (inheritance, genericity, information hiding); Use Design by Contract for reliability in critical systems; Apply fundamental design patterns to solve recurring problems; Read and interpret Eiffel notation for OO concepts; Practice agile development in team settings

Activity: Articulate & Connect | Individual (2 min): Choose one objective and write how it connects to another. Example: 'Information hiding supports Design by Contract by...' Pair (2 min): Share your connection and identify which pillar(s) each objective belongs to (Construction/Architecture/Engineering). Whole class: Volunteers share insights on how objectives build on each other. | Move from remembering objectives to understanding relationships and articulating how course components integrate.

Speaker notes: Present objectives, then facilitate the activity to address the 'articulate course structure' objective at understand/analyze level. Listen for connections like 'information hiding enables reliable contracts' or 'patterns apply inheritance principles.' Push students to explain relationships in their own words rather than repeat bullet points. This prepares them to see how Construction concepts (inheritance, contracts) enable Architecture work (patterns) guided by Engineering practices (agile). Connect their insights to upcoming slides on quality attributes and OO fundamentals.

## Slide 04. The Software Engineering Challenge

Bloom level: analyze | Layout: content

Evidence: Students must analyze relationships between SE challenges and OO solutions, ranking problems by severity and connecting systematic approaches to quality attributes.

heading: Scenario: You inherit a 500,000-line codebase with no documentation

bullets: Scale: 500K lines across 200 files, 15 developers over 5 years; Task: Add a new payment method without breaking existing functionality; Constraints: 2-week deadline, no original developers available

Activity: Analyze the Challenges | Individual (3 min): List 3-4 specific problems you'd face and rank them by severity. Pairs (3 min): Compare lists. Which problems stem from lack of systematic SE practices? How would OO principles (encapsulation, abstraction, contracts) address each? Whole class: Share top challenges and proposed solutions. | Analyze relationships between SE challenges and OO solutions; connect systematic approaches to quality attributes students will learn.

Speaker notes: Present the scenario as real-many students will face this professionally. After the activity, reveal historical examples: Therac-25 radiation overdoses (poor encapsulation), Knight Capital $440M loss in 45 minutes (lack of robustness), Healthcare.gov launch failure (complexity without systematic design). Connect their identified problems to upcoming topics: if they mentioned 'finding where payment logic lives,' that's information hiding (slide 10); 'understanding what code does,' that's abstraction (slide 12); 'ensuring correctness,' that's Design by Contract. The gap between ad-hoc coding and systematic engineering is what this course bridges.

## Slide 05. Software Quality Attributes

Bloom level: evaluate | Layout: content

Evidence: Activity requires evaluating trade-offs between quality attributes in different contexts and justifying priority rankings, reaching evaluate level.

bullets: Correctness: Meets specification; Robustness: Handles abnormal conditions; Extendibility: Adapts to change; Reusability: Works in different contexts; Efficiency, Portability, Maintainability

Activity: Analyze Quality Priorities | Individual (2 min): You're designing software for TWO systems: (A) aircraft collision avoidance, (B) mobile game. Rank the top 3 quality attributes for each and note why. Pairs (3 min): Compare rankings. What trade-offs emerge? If you optimize for efficiency in system A, which attributes suffer? Class discussion: Justify your rankings and identify conflicts between attributes. | Analyze relationships between quality attributes and context; evaluate trade-offs in real design decisions.

Speaker notes: This activity moves from definitions to analysis and evaluation. Students must recognize that context determines priority: aircraft systems demand correctness and robustness over extendibility; games prioritize efficiency and portability. During discussion, highlight trade-offs they identify: optimizing efficiency often reduces extendibility (tight coupling, specialized code); maximizing reusability may sacrifice efficiency (generalization overhead). Connect their insights to slide 4's scenario: the inherited codebase likely prioritized short-term efficiency over extendibility. This prepares them to see how OO principles (next slides) systematically address these quality attributes.

## Slide 06. The Object-Oriented Paradigm

Bloom level: analyze | Layout: content

Evidence: Students analyze how architectural decisions affect software quality by identifying problems with procedural separation and connecting OO bundling to quality attributes.

quote: Organize software around data (objects) rather than functions

bullets: Objects combine data and operations together; Paradigm shift from procedure-oriented thinking; Supports major quality attributes differently than procedural code

Activity: Analyze the Paradigm Shift | Individual (2 min): Consider a STUDENT_RECORD system with data (name, ID, grades) and operations (enroll, calculate_GPA, update_grade). List 2-3 problems that could arise if data and operations are separate (procedural style). Pairs (3 min): Compare your lists. How would bundling data+operations into a STUDENT object prevent these problems? Which quality attributes improve? Class discussion: Share your analysis and connect to slide 5's quality attributes. | Analyze how architectural decisions affect software quality; apply OO thinking to identify design improvements.

Speaker notes: This activity elevates the slide from understanding OO concepts to analyzing their impact. Students must identify concrete problems with procedural separation: inconsistent state (GPA not updated after grade change), unauthorized access (any code can corrupt ID), scattered validation logic. Then analyze how OO bundling addresses these through encapsulation. During discussion, push them to name specific quality attributes: correctness (consistent state), robustness (controlled access), maintainability (localized logic). This prepares them to see these principles in action on slide 7's code comparison and connects back to slide 5's framework.

## Slide 07. Procedural vs. Object-Oriented

Bloom level: analyze | Layout: split

Evidence: Activity requires analyzing relationships between design decisions and quality attributes, determining which attributes are compromised and why.

code: {"language": "pseudocode", "code": "// Procedural: Separate data and functions\nbalance: REAL\n\nprocedure deposit(amount: REAL)\n balance := balance + amount\nend\n\nprocedure withdraw(amount: REAL)\n balance := balance - amount\nend"}

code: {"language": "eiffel", "code": "-- Object-Oriented: Bundled data and operations\nclass BANK_ACCOUNT\n balance: REAL\n \n deposit(amount: REAL)\n do\n balance := balance + amount\n end\n \n withdraw(amount: REAL)\n do\n balance := balance - amount\n end\nend"}

Activity: Analyze Quality Impact | Individual (2 min): Identify 2-3 specific problems that could occur with the procedural approach (consider: who can access balance? what invalid operations are possible?). Pairs (3 min): For each problem, determine which quality attribute from slide 5 is compromised (correctness, robustness, maintainability, etc.) and explain why the OO approach prevents it. Class discussion: Share findings and connect encapsulation to quality. | Analyze relationships between design decisions and quality attributes; apply information hiding principles to evaluate designs.

Speaker notes: This activity moves students from passive comparison to active analysis. Expected problems: balance exposed globally (any code can corrupt it - robustness failure), no validation possible (can set negative balance - correctness issue), changes require finding all balance references (maintainability problem). The OO approach encapsulates balance, enabling controlled access and validation. Push students to explicitly name quality attributes and justify their reasoning. This directly addresses the learning objective to recognize relationships between OO decisions and quality attributes, elevating from understand to analyze level.

## Slide 08. Objects and Classes

Bloom level: apply | Layout: split

Evidence: Students apply class design principles to create a STUDENT class, making abstraction decisions about what to include/exclude.

bullets: Class: Blueprint/template defining structure and behavior; Object: Runtime entity with unique identity and state; Instance: Specific object created from a class; Each object maintains independent attribute values

diagram: UML-style diagram showing BANK_ACCOUNT class box with attributes (balance: REAL, account_id: STRING) and methods (deposit, withdraw, get_balance). Below, show three instances: 'acc1: BANK_ACCOUNT' with balance=1000.0 account_id='A001', 'acc2: BANK_ACCOUNT' with balance=2500.0 account_id='A002', 'acc3: BANK_ACCOUNT' with balance=150.0 account_id='A003'. Dashed arrows from instances to class.

Activity: Design Your First Class | Individual (3 min): Design a STUDENT class for a university registration system. Determine 3-4 essential attributes and 2-3 key operations. Consider: What data identifies a student? What behaviors are needed? Pairs (3 min): Compare designs. Justify why you included each element and what you excluded (apply abstraction). Sketch 2 example instances with different attribute values. | Apply class design principles and abstraction to create a new class; distinguish class structure from instance state.

Speaker notes: This activity elevates the slide from understand to apply/create level, addressing the gap in applying abstraction and information hiding principles. Students must make design decisions about what to include/exclude, applying abstraction in a concrete context. Comparing designs reveals different valid abstractions based on system context. Expected attributes: student_id, name, major, credits_earned. Expected operations: enroll, drop_course, calculate_gpa. Push students to justify exclusions (e.g., why not include home_address if registration system doesn't mail anything?). The instance sketching reinforces class-vs-object distinction while applying the concept.

## Slide 09. Encapsulation: Bundling Data and Operations

Bloom level: apply | Layout: content

Evidence: Activity requires applying encapsulation principles to reorganize scattered shopping cart code into a cohesive class structure.

bullets: Group related data and operations into one unit; Data and methods kept together in the class; Prevents scattered code from manipulating data; Foundation for modularity and coherent design

diagram: Visual representation of a STACK class as a rounded rectangle boundary. Inside show 'items: ARRAY' and 'top: INTEGER' as data, and 'push()', 'pop()', 'is_empty()' as operations. Outside the boundary, show client code with arrows pointing only to the operations, not the data.

Activity: Apply Encapsulation (4 minutes) | Individual (3 min): You're refactoring a shopping cart system where cart_items (array), total_price (real), and discount_rate (real) are global variables, with add_item(), remove_item(), and calculate_total() as standalone functions scattered across files. Redesign this as an encapsulated SHOPPING_CART class. What goes inside the boundary? Pairs (1 min): Compare designs and identify which quality attributes improve. | Apply encapsulation principles to reorganize scattered code; connect design decisions to quality attributes.

Speaker notes: This activity moves from understanding encapsulation to applying it in a redesign scenario. Expected solution: all three data elements and three operations bundled inside SHOPPING_CART class boundary. After activity, highlight quality improvements: maintainability (all cart logic in one place), extendibility (easy to add new cart features), robustness (prevents external code from creating inconsistent states). Connect to slide 5's quality framework. Use toolbox analogy: scattered tools vs. organized toolbox sections. This prepares students for information hiding in slide 10, where we'll add public/private distinctions.

## Slide 10. Information Hiding: Interface vs. Implementation

Bloom level: apply | Layout: content

Evidence: Students apply information hiding by redesigning an EMAIL class, deciding what should be public interface vs. private implementation.

bullets: Public interface: What clients can use (contract); Private implementation: How it works internally; Clients depend only on interface, not implementation; Enables changing implementation without breaking clients

code: {"language": "eiffel", "code": "class STACK\nfeature -- Public interface\n push(item: INTEGER)\n pop: INTEGER\n is_empty: BOOLEAN\n\nfeature {NONE} -- Private\n items: ARRAY[INTEGER]\n top: INTEGER\nend"}

Activity: Apply Information Hiding (4 minutes) | Given this poorly designed EMAIL class with all features public, redesign it applying information hiding. Decide what should be public interface vs. private implementation. Justify which quality attributes improve and why.

Speaker notes: Give 3 minutes for redesign, 1 minute for pair discussion. Expected solution: send() as public interface; validation and formatting as private helpers; data attributes private with query methods if needed. This improves extendibility (can change validation rules), maintainability (centralized logic), and robustness (prevents invalid states). After activity, show one strong solution and discuss how information hiding directly supports the quality attributes framework from slide 5. Use ATM analogy: interface is buttons, implementation is banking backend.

## Slide 11. Analyze: How Encapsulation Supports Quality

Bloom level: analyze | Layout: content

Evidence: Explicit analysis task requiring students to examine how two designs support or violate specific quality attributes with justification.

text: Consider two BANK_ACCOUNT implementations:

**Design A**: Public balance attribute, clients directly modify it
**Design B**: Private balance, only deposit/withdraw methods can change it

Activity: Analysis Task (3 minutes) | For each design, analyze: Which quality attributes does it support or violate? Consider correctness, robustness, extendibility, maintainability. What specific problems could arise? Be prepared to justify your analysis.

bullets: Correctness: Can invalid states occur? How?; Robustness: Where is validation logic?; Extendibility: Can implementation change safely?; Maintainability: Where are changes localized?

Speaker notes: Give students 2 minutes individual thinking, 1 minute pair discussion. Design A allows negative balances, scattered validation, brittle clients. Design B prevents invalid states through controlled access, centralizes validation, enables changing storage mechanism without breaking clients. After activity, reveal that Design B exemplifies how encapsulation directly supports quality. Ask volunteers to share one quality attribute they analyzed. This moves from being told relationships to discovering them through analysis.

## Slide 12. Abstraction: Focusing on the Essential

Bloom level: apply | Layout: content

Evidence: Students apply abstraction principles by designing VEHICLE classes for different contexts, justifying inclusion/exclusion decisions.

bullets: Focus on essential properties for a specific purpose; Different contexts require different abstractions; Determines what to include/exclude in a class; Same entity, multiple valid abstractions

Activity: Apply: Design a VEHICLE Class

diagram: Two side-by-side class boxes for PERSON. Left labeled 'HR System' shows: employee_id, name, salary, department, hire_date. Right labeled 'Medical System' shows: patient_id, name, blood_type, allergies, emergency_contact. Caption: 'Same entity, different essential properties'

Speaker notes: This activity directly addresses the 'apply abstraction principles' objective by requiring students to make abstraction decisions for new contexts. During pair comparison, students discover different valid choices. Rental system likely includes: mileage, fuel_level, rental_rate, daily_cost, is_available. Repair shop includes: VIN, make, model, engine_type, service_history, parts_needed. Guide discussion: Why exclude color from repair shop? Why exclude engine_type from rental? Emphasize that abstraction isn't about hiding implementation-it's about choosing what matters for your problem domain. Connect to quality: wrong abstraction level creates unnecessary complexity or missing functionality.

## Slide 13. Abstract Data Types: Interface vs Implementation

Bloom level: analyze | Layout: content

Evidence: Activity requires analyzing trade-offs between PRIORITY_QUEUE implementations against multiple quality attributes (performance, memory, complexity).

bullets: ADT defined by operations (interface), not representation; QUEUE: enqueue, dequeue, is_empty, count; Multiple implementations possible: array, linked list, circular buffer; Clients depend on abstraction, not implementation details

Activity: Apply: Design Decision Analysis

diagram: Simple layered diagram: top layer shows QUEUE interface box with operations (enqueue, dequeue, is_empty). Arrow pointing down to three implementation boxes side-by-side labeled 'Array', 'Linked List', 'Circular Buffer'. Dotted line separates interface from implementations.

Speaker notes: This activity targets the 'apply information hiding and abstraction' objective by requiring students to analyze trade-offs between implementations. Guide discussion toward: sorted array gives O(n) insert but O(1) removal; heap gives O(log n) for both. Memory efficiency and complexity differ too. The key insight: clients using the ADT interface don't need to know implementation details, but designers must analyze quality attribute trade-offs when choosing implementations. This connects ADT abstraction to quality attributes through practical analysis.

## Slide 14. Why Eiffel? Applying Notation to Express Design

Bloom level: apply | Layout: split

Evidence: Students apply Eiffel syntax to write a CIRCLE class while simultaneously applying information hiding principles.

bullets: Pure OO language designed for software quality; Clean syntax explicitly expresses OO concepts; Built-in Design by Contract support; Created by Bertrand Meyer (course coordinator); Concepts transfer directly to Java, C++, Python, C#

Activity: Apply: Write Your First Eiffel Class | On paper, design a CIRCLE class in Eiffel with: (1) radius attribute, (2) make constructor accepting radius value, (3) area query function, (4) circumference query function. Use proper Eiffel syntax: class/feature/end keywords, create clause, Result for queries. Then compare with a partner: did you apply information hiding by making radius a feature, or hide it as implementation?

Speaker notes: This activity elevates the slide from understand to apply level, addressing the gap in Eiffel notation application. Students apply syntax rules to create a new class while simultaneously applying information hiding principles by deciding what to expose. The pair comparison surfaces the design decision: should radius be publicly accessible or hidden with only computed properties exposed? This connects Eiffel notation to the information hiding objective. Circulate to check syntax (common errors: forgetting end, using = instead of :=, omitting Result). Debrief quickly: both designs are valid, but hiding radius increases flexibility for future changes (e.g., storing diameter instead).

## Slide 15. Basic Eiffel Class Structure

Bloom level: apply | Layout: split

Evidence: Activity requires applying Eiffel syntax rules to write a RECTANGLE class using POINT as a template.

code: {"language": "eiffel", "code": "class POINT\n\ncreate\n make\n\nfeature -- Initialization\n make(x_val, y_val: REAL)\n do\n x := x_val; y := y_val\n end\n\nfeature -- Access\n x, y: REAL\n \n distance: REAL\n do\n Result := sqrt(x^2 + y^2)\n end\nend"}

Activity: Apply: Write Your First Class | On paper, write an Eiffel RECTANGLE class with: (1) width and height attributes, (2) a make constructor, (3) area and perimeter query functions. Use POINT as your template. Focus on correct syntax: feature clauses, Result for return values, := for assignment.

Speaker notes: First show the POINT structure: class declaration, create clause for constructor, feature clauses grouping related operations, make procedure with parameters, attributes x and y, query function using Result. Then launch the activity. This moves from understanding notation to applying it. Common mistakes: forgetting 'end' keywords, using = instead of :=, omitting Result. Circulate briefly, then have pairs discuss challenges. This addresses the gap in applying Eiffel notation and supports the information hiding objective by having students determine what to expose as features.

## Slide 16. Eiffel Syntax Essentials

Bloom level: understand | Layout: split

Evidence: Slide presents syntax rules for reading Eiffel code without requiring application or analysis, focusing on comprehension.

bullets: Declaration: variable: TYPE; Assignment: :=; Creation: create obj.make; Feature call: obj.feature_name; Comments: -- single line

code: {"language": "eiffel", "code": "-- Variable declaration\nmy_point: POINT\n\n-- Object creation\ncreate my_point.make(3.0, 4.0)\n\n-- Feature call\ndist := my_point.distance_to_origin\n\n-- Output\nprint(dist) -- prints 5.0"}

Speaker notes: These are the basics you need to read Eiffel code. Key differences from Java/Python: := for assignment (not =), type comes after variable name, create keyword for instantiation. The syntax is very regular and consistent. In tutorials, you'll practice writing Eiffel code. For now, focus on being able to read and understand it. The examples in upcoming lectures will use this notation.

## Slide 17. Course Roadmap and Design Challenge

Bloom level: apply | Layout: split

Evidence: Synthesis activity requiring students to apply abstraction and information hiding to design a STUDENT class with justified choices.

bullets: Lectures 1-3: OO Fundamentals (classes, encapsulation, abstraction); Lectures 4-7: OO Mechanisms (inheritance, polymorphism, genericity); Lectures 8-12: Design Patterns (reusable solutions); Lectures 13-16: Software Engineering (agile, requirements)

Activity: Apply: Design a Student Class

Speaker notes: This activity synthesizes today's concepts by requiring students to apply abstraction and information hiding to create a new design. They must determine what's essential for a STUDENT class, decide encapsulation boundaries, and justify choices using quality attributes. This moves from analyzing existing designs to creating new ones. Circulate during pair-share to identify common decisions. Debrief by highlighting how different contexts (enrollment system vs. library system) would lead to different abstractions, previewing how these fundamentals support upcoming topics on inheritance and patterns.

## Slide 18. Summary and Next Steps

Bloom level: remember | Layout: content

Evidence: Recap of key points for retention without requiring higher-order cognitive processing.

heading: Key Takeaways

bullets: Software engineering provides systematic approaches for quality; OO organizes software around objects (data + operations); Encapsulation and information hiding enable quality attributes; Abstraction focuses on essential characteristics; Eiffel provides clean notation for OO concepts

text: Next Lecture: Inheritance and Polymorphism

text: Preparation: Install EiffelStudio, complete intro exercises, read Touch of Class Ch. 1-3

Speaker notes: Recap the main points briefly. Emphasize that today laid the conceptual foundation - everything else builds on these principles. For next lecture, they should have EiffelStudio installed and have experimented with basic class creation. Encourage questions and remind them of office hours. End on an encouraging note: these concepts may seem abstract now, but will become concrete through practice in tutorials and assignments.